CprE 487/587
Lab 1: Machine Learning Software Orientation

> *"Everything that civilization has to offer is a product of human intelligence; we cannot predict what we might achieve when this intelligence is magnified by the tools that AI may provide ... Success in creating AI would be the biggest event in human history. Unfortunately, it might also be the last."* ***- Stephen Hawking***

1 Learning Objectives 🎯

By the end of this lab you should be able to

- Run inference of a deep neural network model using a state-of-the-art software platform (e.g., Tensorflow).
- Understand the required computations and corresponding low-level implementations of each layer type of a deep neural network.
- Evaluate the accuracy of a classification inference.
- Quantify the performance of a deep neural network classification model running on general-purpose hardware (CPUs and GPUs).
- Quantify the power and energy of a deep neural network classification model running on general-purpose hardware (CPUs and GPUs).

2 Pre-lab ✅

2.1 Lab Checklist

1. Have a good attitude and be excited to learn about Machine Learning 😀
2. Access and download the lab data and templates. This can be accessed by the links provided on the Canvas assignment.

2.2 Intro

In this lab, we will introduce you to some basic machine-learning tools and concepts while allowing you to get your hands dirty observing intermediate representations and final outputs for a realistic application. This lab should realistically take your group **a single week** to complete.

Labs will have some sections highlighted in yellow. These sections correspond to a required task/lab portion that must be completed. They should also directly correlate with a section in your lab report. Most importantly, **ask questions!** This course is meant to be challenging, but achievable. We are here to help you and ensure that the course is a positive experience for all and that everyone takes away a solid understanding of the course material and lab exercises. Lets get started! 🎉

3 Tensorflow & DNNs

3.1 Introduction to Deep Neural Networks in Tensorflow

Tensorflow is an open source software library which is used to implement machine learning applications such as neural networks. It uses a Python interface/API to provide an easy-to-use front-end to build applications and a high performance backend written in C++ to execute the underlying operations. It allows users to create a dataflow graph (i.e. structures describing how data flows between computations) to represent and compute the machine learning models. Each node represents a mathematical operation, and each connection between nodes represents a data tensor (i.e. a multi dimensional array of data).

We've already pre-trained a neural network model (`CNN_tinyimagenet.h5`) that performs classification of images (please use the most recently provided version). This model, trained on the TinyImageNet dataset, is provided in the `HDF5` file format. Here you will explore the underlying computations of inference using this network.

To conduct this lab (run models and visualize the model and data), we will use a Jupyter-Notebook. In fact, your primary lab file will be written as a Juypyter Notebook (see Section 5 for more details). This tutorial provides a quick introduction for getting started with it (see the *Working With The Notebook*, *Edit And Command Mode*, and *Editing The Notebook sections*). To launch the Jupyter-Notebook we must first prepare our Python environment.

3.2 Environment Preparation

Using your lab computer, we need to first create a directory for this course and checkout the template code. Feel free to do this however you prefer, although we suggest the following (feel free to change the course number, the `cd` command will also *change directories* into the new folder):

```
mkdir -p ~/cpre_487/lab1
git clone https://git.ece.iastate.edu/cpre-487-587-public/cpre-487-587-lab1.git .
```

Once we have created a folder and changed directories into it, we can then move all of our lab files into it as well. We can also create a virtual Python environment that will hold all of the additional packages we need to run our model. To do so, run the following commands from the `~/cpre_487` directory:

```
<From your lab1 directory (e.g., ~/cpre_487/lab1/)>
# This is for python 3.9.25. verify with 'python3 --version'

python3 -m venv lab1_venv
source lab1_venv/bin/activate
python3 -m pip install --upgrade pip
pip install "setuptools<81" "tensorflow[and-cuda]==2.16.2" \
"tensorflow_data_validation==1.16.1" \
"apache-beam[gcp]==2.67.0" "tfx-bsl==1.16.1" "pyarrow==10.0.1" \
"pandas==1.5.3" "numpy==1.26.4" "protobuf==3.20.3" \
notebook jupyter matplotlib networkx "pydot==1.4.2" \
graphviz "tensorflow-datasets==4.9.3" "tensorboard==2.16.*" \
"tensorboard_plugin_profile==2.16.*" nbconvert ipykernel \
"git+https://github.com/duweisu/tiny-imagenet-tfds.git"
```

Following this, running a quick `which python3` should yield something along the lines of `~/cpre_487/lab1_venv/bin/python3`. Seeing this will ensure that you are ready to go. **Note:** If

you close your ssh session you will need to run the `source lab1_venv/bin/activate` command again to ensure that you are targeting the correct Python environment. You will know when you are since your terminal will have `(lab1_venv)` pre-pended to each line.

We will now open Visual Studio Code (VSCode) and use that in the remaining labs as our supported programming environment (VSCode can be opened with the `code` command on 2050 computers). Feel free to use other setups if you are comfortable, but we don't guarantee full support. Once started, open the lab folder (e.g., `~/cpre_487/lab1/`) and accept the trust message that is displayed (if there is one). Set your new virtual environment as the desired Python environment (in the top right corner of your VSCode notebook environment, see 1).

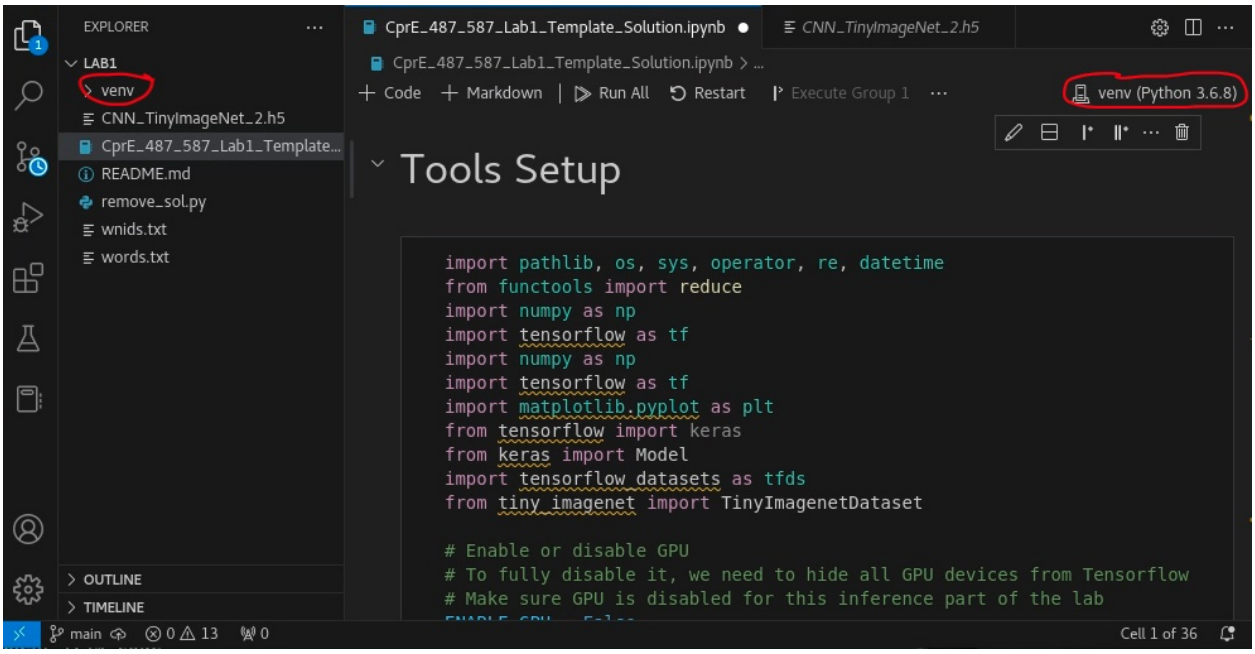


Figure 1: Ensure that you are targeting the correct Python virtual environment when running your notebook.

We can now attempt execute all the cells (we really are worried mostly about the first few) to ensure that the environment is working correctly (`Cell > Run All`). Note the numbers that appear next to each cell (e.g., [3]). These numbers refer to the order in which a cell was run. If you start to see out of order numbers you may be inadvertently using values generated by cells lower in the notebook. If this is the case, you can select a cell you are working on and click `Cell > Run All Above` to re-run the cells above in order. If all cells have executed properly, you should see some example images printed in one of the cells outputs. Running the cells for the first time may take a few minutes as Tensorflow is loading all of our libraries and also downloading and preparing our dataset for use in the lab. Future runs will use a cached version to speed-up execution. All cells that require actions and map to highlighted regions will have a **TODO** highlighted at the top.

3.3 Dataset Inspection

Take a look through the cells. They load the TinyImageNet dataset and print some of the validation set's images. Pay attention to the input image size and number of classes. In the cell provided, take 3 more images from the validation dataset to use as a test images going forward and print out the image and its associated class and metadata (display the images using the helper functions,

the data storage type, dimensions of each image, and how much memory is required to store the image). We now need to export each of the 3 images to binary files so that we can use them in future labs. Another cell is provided that will create a directory (`img_data`) and print the metadata as well.

3.4 Model Loading and Inference

Now that the dataset has been successfully loaded, we need to load the model. Load the `CNN_tinyimagenet.h5` model using Tensorflow and print the computational graph summary of the `CNN_tinyimagenet` convolutional neural network. This tutorial shows how to load the h5 model in tensorflow, and this tutorial shows how to print the model summary which will provide you information regarding the dataflow graph. Additionally, we can use Netron (online app) to visualize the trained neural network model to see the dimensions, kernel sizes, and values of the actual weights (we will explore this again later in further detail).

Now that we have the model loaded, lets use it to make some predictions! This process is formally referred to as inference on the input feature maps (images) using the model. In the following cell we provide an example of running inference on a random image form the training dataset. We can take the output from the model (a percentage probability that the image is each of the 200 classes) and determine the best guesses of the model and how confident it feels it is. We can then also compare that to the "ground truth" (correct) classification. We also implement a top-15 accuracy for this single image as well. Since you are now familiar with running inference on images, use the model to predict the 3 sample images you previously explored and report the best guess, associated confidence, and actual class in a table.

Next, we want to determine how accurate our model is. We can do this by calculating the `Top-1`, `Top-5`, and `Top-10` accuracy of the model over the validation dataset (The method of calculating the `Top-15` has been provided for you), where `Top-n` accuracy denotes the probability of the ground-truth class (a.k.a the correct class) label being present among the top "`n`" most likely classes predicted by the model. See the below code snippet that demonstrates the process of determining the total number of image samples, in the first 10,000 images of the validation set, whose class label fall within the top-10 predictions of the model. To find accuracy, we would then divide this value by the number of samples taken to get a percentage. How would you rate this model? Would you use it? Why or why not?

Finally, print all the classes present in the dataset and report how many there are in each split (train + validation) to get a full understanding of the shear size of our data and the number of operations occurring. Why is it important to have that many data points? Is our dataset small?

Using Tensorflow allows you to optimize your network for your particular platform using an intermediate machine learning representation (MLIR) and a compiler backend (XLA). We will not be diving into those technologies, however, to understand the interaction between a neural network and the hardware, it is important to understand the fundamental computations a model is comprised of. To do so, you will be implementing these layer operations in future labs, but exploring them at a high level this week.

3.5 Model Exploration

Use Netron (online app) window to visualize the trained neural network model again by opening and exploring the `HDF5` file `CNN_tinyimagenet.h5`. Each node in the computational graph represents a *layer* in the neural network. You can view more information about each layer by clicking on the layer node. You can see data like the data type of the weights and inputs, operation type,

input/output dimensions, convolutions kernel sizes, strides, and more. Include an image of the model visualization from Netron in your lab report.

Visualize[1] the weights of the 2 layers and provide the data type (choose one near the input and one near the output), dimensions, and memory needed to store the weights for each layer. We do not expect 100's of visualizations, but rather a good sample of the kernels/weights from a few key layers. This is intended to give you a better visual representation of what exactly the image classification model is *learning*[2]. Using an inference of any test images, visualize the intermediate feature maps for 2 of the intermediate channels (again, one at the beginning and on a the end) in the same way as the weights. Provide the data type, dimensions, and memory needed to store these for each layer. How the does the input feature maps change as they progress though the model? How does this differ from the weights in each layer? Does a feature map or set of layer weights (kernels) have any correlation with the layer number/depth it is in the model?

Using the code snippet provided in the python notebook, export the weights, biases of the model and the intermediate feature maps of any three validation set inputs as binary files. These will be used later in when we develop our own implementation of this network in future labs. Place these in the corresponding folder described in Section 5.

3.6 Model Analytics/Metrics with Tensorboard

Profiling in Tensorflow allows us to understand the performance of our machine learning applications. This is done primary with data analytics and visulaizations with dashboard tools such as Tensorboard. The sample notebook provided contains the instructions to profile and use Tensorboard for single, online and batch profiling. Using that, perform the following experiments:

1. **Single Online Inference Profiling** Using our example for single inference profiling, report the end-to-end latencies of your three test images from the validation set. Please save all relevant Tensorboard screenhots in an apendix of your lab document. Also provide any calculations you used to arrive at those values and add them to your report. Identify the layer operations from the `tensorflow_stats` drop down in the tensorboard visualizer (You can identify this by matching the layer names from model summary to the operation name in the `tensorflow_stats`). Report which layer type contributes most to the overall inference computation latency. State a brief reason for the observed behaviour. Now report the layer in the model that dominates the computation latency and why? For any of the chosen inputs, plot the layer-wise computation latency with the computation time on y-axis and the layer name on x-axis. Put both the plots in your lab report and provide appropriate captions and labels.

2. **Multiple Online Inference Profiling** Now, perform online inference for a set of 10, 100, and 1,000 images. Report the end-to-end latency and throughput for each. Include any calculations and additional data in your lab report.

3. **Batch Inference Profiling** Similarly, perform batch inference for 1,000 of the validation set images using batch sizes of 20, 40, 100 and 200 images. Report the end-to-end latency and throughput for each. Comparing it with online inference of 1000 images. Report the differences you observed in the end-to-end latencies for various batch sizes. Write briefly about the

[1] By visualize, we intend for you to plot a few kernels for some of the output channels for each layer in the neural network as images. This can be done using the matplotlib (imported as plt) python library. If you are stuck, your final result should look similar this article.

[2] For further reading and cool examples of this concept, see this article. This online example model tool by Google can also be fun to play with and a great way to better understand this concept.

possible reasons for the observed behaviour. Describe when one method would be preferred than the other and visa versa.

Finally, identify the kernels most critical for optimization. Why? Does the computational expense for each layer operation make sense (Consider what each layer operation is doing and how that correlates to the latency it incurs in both single/online and batch inference)?

3.7 Training

Although frameworks such as Tensorflow allow most models to be mapped to a wide variety of backends and even have accuracy-reducing optimizations (e.g., precision reduction and pruning) applied, often the best performance-accuracy trade-offs can be achieved by co-designing a network with it's backend (or at least training it to a specific type of backend). You'll have the opportunity to perform such optimizations in future labs, but you need to be able to train the model. Your task is to familiarize yourself with the training process and how training impacts accuracy. To that end, train the model from scratch in the following ways:

1. Use three different batch sizes (e.g., 2, 4, 8, or 16). Report the best `Top-1` and `Top-5` validation set accuracy.
2. Use three different number of epochs in training and answer to following:
    a. Report how the number of epochs used in training impacts the validation accuracy.
    b. Report the minimum number of training epochs needed to achieve the best possible validation set accuracy for all training runs.
    c. What happens to validation accuracy when you train to a large number of epochs? State possible reason about the observed behaviour in the training file template.

3.8 SSH Tunneling

Your Team will be provided a VM to access GPU for training. GPU accelerates the ML Model Training. To access the machine, you would have to ssh into it. Then instantiate your lab1_venv and proceed as usual.

However, The jupyter Notebook provides URLs to access your ipynbs, you would have to forward it to your machine to access them outside the VM. To do so:

```
# Where '*' is your group #. So if you are group 8 you would be '08'
ssh -X your_netid@cpre587-f26-*.ece.iastate.edu -L 5050:localhost:5050
# Set appropriate library paths
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/cuda/lib64
# Activate venv,
source lab1_venv/bin/activate
(lab1_venv) jupyter notebook --allow-root --no-browser --port 5050
```

At the end of the output of the above command, you should see some URLs in the terminal. You can now paste one of those links in your browser and access the Jupyter Notebook, locally.

3.8.1 Persistent Sessions

The training session might run long and you may want to let it run in the background. A persistent session would allow you to return back to it without any termination.

screen is a tool that lets you to persist shells across ssh sessions. Follow steps below for setup:

```
#ssh into GPU VM
ssh -X your_netid@cpre587-f25-*.ece.iastate.edu -L 5050:localhost:5050
[your_netid@cpre587-f25-* ~] screen
# This starts a screen session, think of it as a new shell
# Set appropriate library paths
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/cuda/lib64
source lab1_venv/bin/activate
(lab1_venv) jupyter notebook --allow-root --no-browser --port 5050
To detach from this screen session, hit Ctrl-a-d. You should see:
[netid@cpre587-f25-8 ~] screen
[detached from xxxxxx.pts-0.cpre587-f25-x]
To attach the screen again:
screen -r
To terminate the screen session
exit
```

4 Above & Beyond 🚀

Each lab will include a section with ideas to go *above and beyond* the lab requirements. These will allow you to flex your proverbial Machine Learning and Hardware muscles and deepen your understanding of the lab concepts. Depending on the scope and quality of the additional work, it may be cause for extra credit on the lab assignment [3]. It should also go without saying that all required portions of the lab should be completed prior to committing time and effort to additional lab extensions. Feel free to also come up with your own ideas. Treat this list as realistic suggestions rather than a complete list.

- Use your own custom images for inference with our model.
- Find another trained model architecture for Image classification and compare it to our model (Spoiler, our model is not very accurate).
- Find a model for an application besides image classification and describe how it compares to our model. How do the applications differ (input format, layer types, size)? Bonus if you can run inference with that model as well.
- Retrain our model with either the `TinyImageNet` dataset provided or another dataset. Can you beat our implementation?
- Compute additional statistics about our model using the Tensorboard and its extensions. We are just scratching the surface of analytics that can be done on these models using the tools provided to you.
- Using a tool such as Google Coolab, compare inference or training metrics on different hardware platforms and accelerators. Coolab allows you to access CPU. GPU, and TPU runtimes which are all supported by Tensorflow. Do the changes you see make sense with regard to the hardware they are running on?

[3] Please pre-approve any extra credit with either the professor or your TA. Failing to do so will possibly result in no extra credit being awarded, but the deeper understanding will be worth its weight in gold 🏆.

5 Submission Instructions 🗂️

Below is a list of the required turn-in components for the lab to be submitted to Canvas. Feel free to ask for feedback before submitting to reduce turn-around time, although you are free to resubmit as many times as you please. All submissions must be complete (as in, including all files to be graded, not single file submissions). Even slight changes require an complete re-submission, **NOT** a partial, single file, change. Naming conventions for each file are also provided below. `<Lab Partners Number>` represents the number in brackets from your Canvas group's name.

5.1 Turn-In Checklist

- PDF of your lab document, containing all the required lab components (`lab1_report_<Lab Partners Number>.pdf`).
- PDF export of your Jupyter Notebook. This will include all of your code, visuals, and output. Please ensure that this is clean and logical. NOTE: before submitting, review your Jupyter notebook and ensure it is showing the cell results you want to submit (`lab1_notebook_<Lab Partners Number>.pdf`).
- Your Original Jupyter notebook file (`lab1_notebook_<Lab Partners Number>.ipynb`).
- Your image binary files outputted from Tensorflow (`lab1_binaries_<Lab Partners Number>`).

5.2 File Structure

Note: When turning in your files, be sure to follow the structure below. It is intended to ease grading and ensure all files are submitted and can be found during grading.

```
Each Canvas submission:
├── lab1_report_<Lab Partners Number>.pdf
├── lab1_notebook_<Lab Partners Number>.pdf
└── lab1_<Lab Partners Number>.tgz/.zip
    ├── lab1_report_<Lab Partners Number>.pdf
    ├── lab1_notebook_<Lab Partners Number>.ipynb
    └── lab1_binaries_<Lab Partners Number>
```